

return value of a function, the address of the next instruction to be executed, etc. These registers exist for a program only as long as the program is running, or if it has not exited normally. Once the program exits, these registers cease to exist.

## Preventing buffer overflow attacks using GDB

In this section, we will look at how to exploit some code that is vulnerable to buffer overflow. GDB plays a very important role here, because it helps us break the code into different segments and inspect the memory in the program.

As an illustration, here is an example of code that is vulnerable to buffer overflow:

```
//example1.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

main ( int argc, char ** argv ) {
    char buf [400];

    if ( argc < 2 ) {
        printf ( "Please enter a command-line argument.\n" );
        exit (0);
    }
    strcpy ( buf, argv[1] );
    return 0;
}
```

This code is vulnerable to buffer overflow because the `strcpy` function does not check whether or not the number of bytes being copied to the buffer is less than the size of the buffer. Before beginning the demonstration, let us understand the layout of the stack with respect to this program. Figure 1 shows you the stack layout. The stack grows 'up' towards the lower memory, which means that as more items get added to the stack, they get stored at lower memory addresses.

When a function is called, the arguments are pushed onto the stack, followed by its return address and frame pointer, and then its local variables. The diagram shows the stack for the `main` function in the code, after space has been allocated to the local arguments as well. In this program, the `strcpy()` function copies the command-line argument passed to the program directly, without checking the number of bytes. As a result, if the user were to give an input string of length greater than 400 (in reality, it is a little more than 400, as will be seen later), the buffer overflows, and the memory addresses adjacent to the buffer (`ebp` and `eip`), get overwritten.

## Compile and run

Now let us compile and run this code. Recent releases of operating systems have inbuilt security measures to protect the stack from getting attacked. The GCC Stack-Smashing

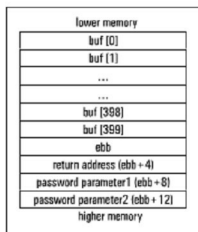


Figure 1: The layout of the stack

Protector (also known as ProPolice) has, by default, got flags set to protect the stack from getting attacked. In order to disable these mechanisms, we give extra arguments while compiling. The argument 'execstack' is given to make the stack executable, that is, the stack can execute the instructions stored in it.

```
gcc -fno-stack-protector -z execstack example1.c -o example1
./example1 `python -c 'print "A" * 400'`
```

## Overwriting the Instruction Pointer (eip)

As long as the command-line argument is less than or equal to 400 bytes, the program works fine. In fact, even for arguments with sizes slightly over 400, the program does not crash, provided the version of the GCC compiler being used is 2.96 and above—because when the program allocates space for the local variables of a function, it subtracts 16 bytes more from the `esp` (stack pointer), than what is required. In the above program, since the `buf[400]` is the only local variable, only 400 bytes need to be allocated. But, in the fourth line of the disassembly, it can be seen that some extra memory space (0x1a016 = 41610) is being allocated. This happens because when compiled with recent versions of GCC, a dummy space of 8 bytes is created after the buffer for other memory registers like `esi` and `edi`. The remaining bytes are for storing the `ebp` (frame pointer) and the return address. Thus, slightly exceeding 400 would not cause a segmentation fault. But for GCC versions below 2.96, no dummy space is created, and exceeding 400 bytes can cause a segmentation fault right away.

```
(gdb) disas main
Dump of assembler code for function main:
0x08048434 <+0>:  push  %ebp
0x08048435 <+1>:  mov   %esp,%ebp
0x08048437 <+3>:  and  $0xffffffff,%esp
0x0804843a <+6>:  sub  $0x1a0,%esp
0x08048440 <+12>: cmpl  $0x1,0x8(%ebp)
0x08048444 <+16>:  jg   0x0804845e <main+42>

<snipped>

0x0804847d <+73>:  call  0x08048340 <puts@plt>
0x08048482 <+78>:  mov  $0x0,%eax
0x08048487 <+83>:  leave
0x08048488 <+84>:  ret

End of assembler dump.
```